

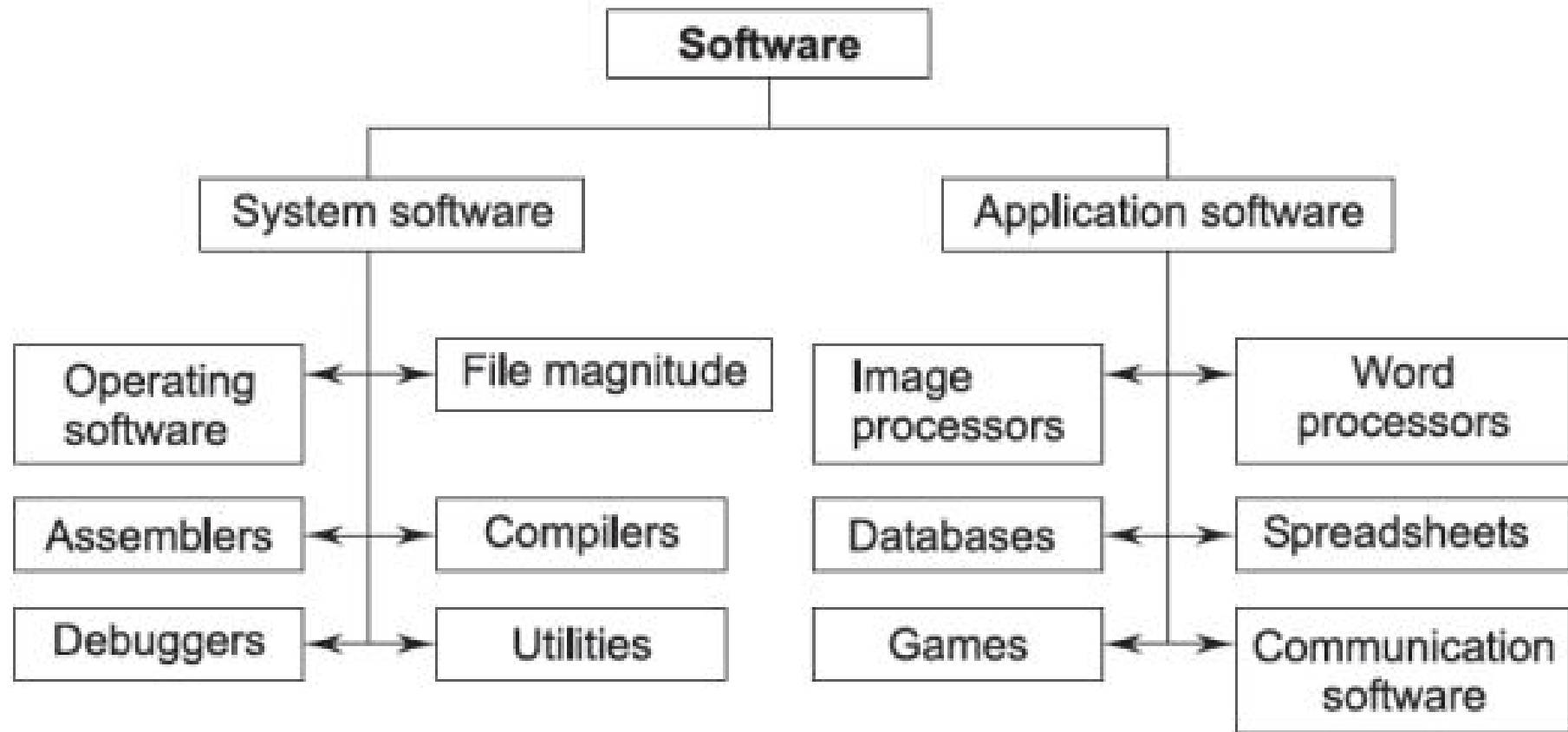


Fundamentals of Software Engineering

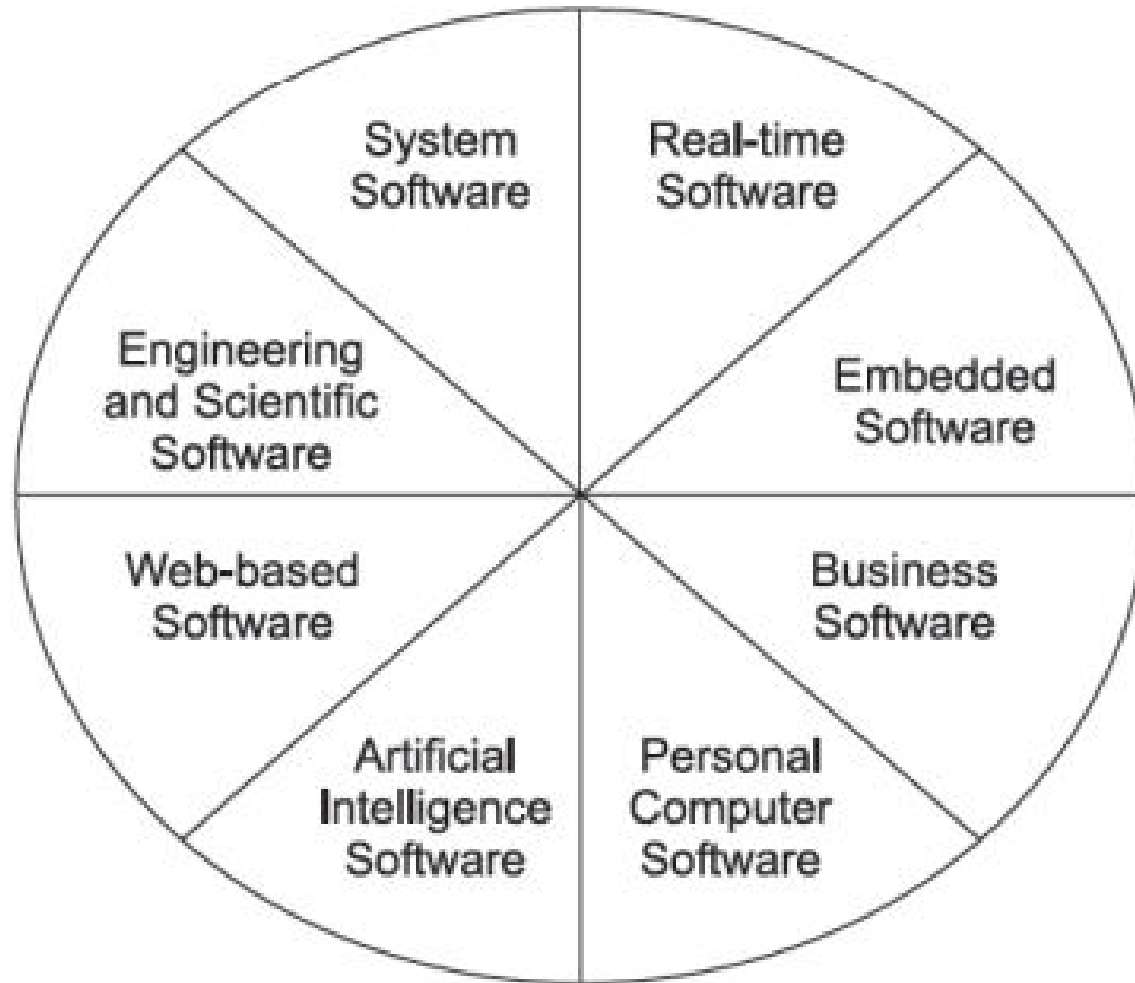
LECTURE 3: SOFTWARE APPLICATION TYPES AND SOFTWARE PROCESSES

Types of Software Application





Types of Software



Software Application Types

Software Application types

Stand-alone applications

- These are application systems that run on a local computer, such as a PC. do not need to be connected to a network. MS Office, Compiler etc.

Interactive transaction-based applications

- Applications that execute on a remote computer and are accessed by users from their own PCs. web applications such as e-commerce applications.

Embedded control systems

- These are software control systems that control and manage hardware devices. Mouse, Network drivers etc.

Application types

Batch processing systems

- These are business systems that are designed to process data in large batches. E.g Credit card companies process bill, generate payroll, generate SMS for clients.

Entertainment systems

- Media players etc.

Systems for modeling and simulation

- These are systems that are developed by scientists and engineers to model physical processes or situations, which include many, separate, interacting objects. CASE, System Arch, Visio etc.

Application types

Data collection systems

- These are systems that collect data from their environment using a set of sensors and send that data to other systems for processing. e.g. RTU in Supervisory Control and Data Acquisition (SCADA) systems

Systems of systems

- These are systems that are composed of a number of other software systems. e.g. Electric/power grid systems

Software Processes

What is a software process?

A set of activities whose goal is the development or evolution of software

Generic activities in all software processes are:

- Specification - what the system should do and its development constraints
- Design and Development - production of the software system
- Validation - checking that the software is what the customer wants
- Evolution - changing the software in response to changing demands



Software Process

Fundamental Assumption:

- Good processes lead to good software
- Good processes reduce risk
- Good processes enhance visibility
- Good processes enable teamwork

Variety of Software Processes

Software products are very varied...

Therefore, there is no standard process for all software engineering projects

BUT successful software development projects all need to address similar issues.

This creates a number of process steps that should be part of all software projects.

What is a software process model?

A simplified representation of a software process, presented from a specific perspective

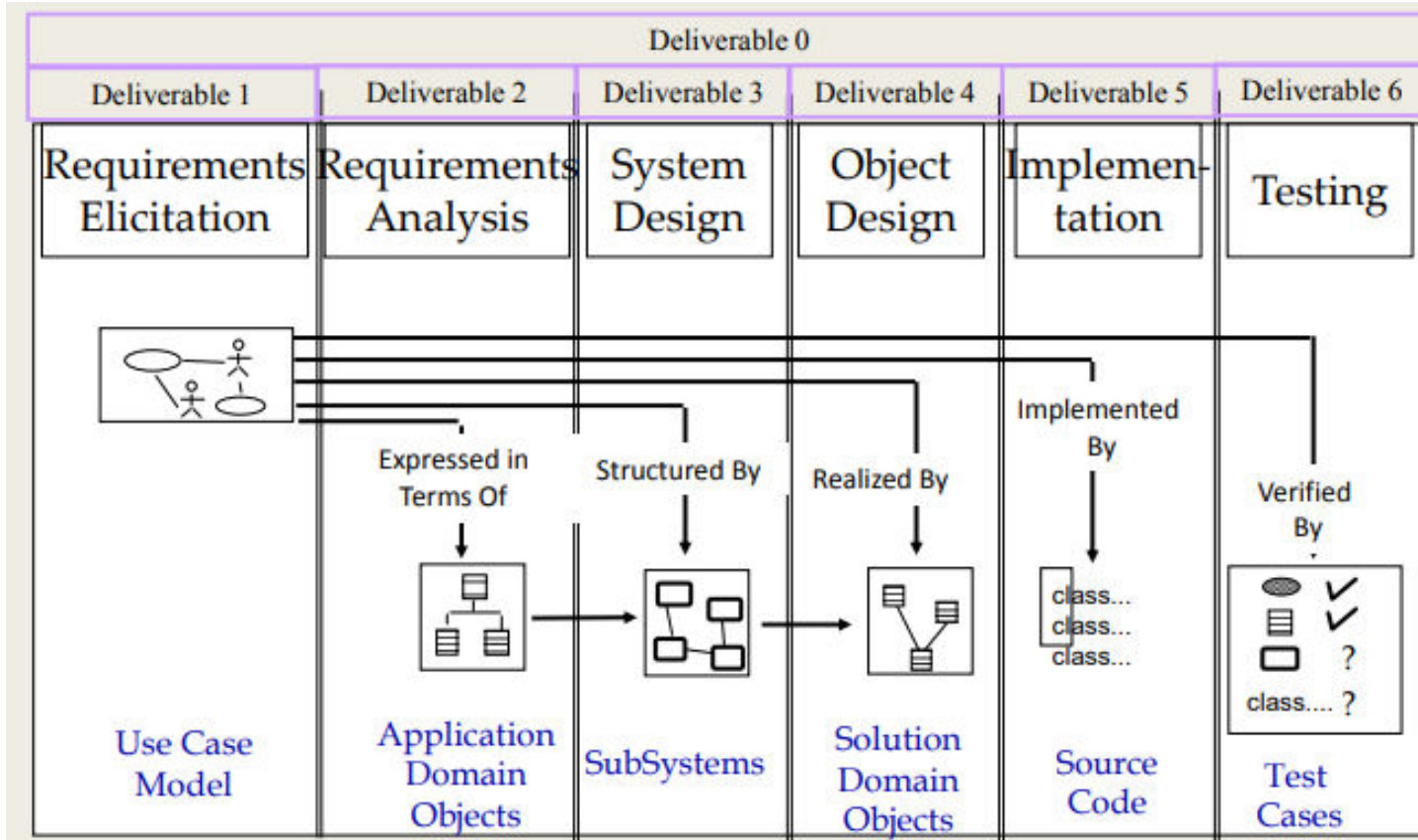
Examples of process perspectives are

- Workflow perspective - sequence of activities
- Data-flow perspective - information flow
- Role/action perspective - who does what

Generic process models

- Waterfall
- Evolutionary development
- Formal transformation
- Integration from reusable components

Software Lifecycle Activities



Basic Process Steps in all Software Development

- Feasibility and planning
-

- Requirements
 - System and program design
 - Implementation
 - Acceptance and release
-

These steps may be repeated many times during the development cycle.

- Operation and maintenance

It is essential to distinguish among these process steps and to be clear which you are doing at any given moment.

Quality Control Steps in all Software Development

- Validating the requirements
 - Validating the system and program design
 - Usability testing
 - Program testing
 - Acceptance testing
-

- Bug fixing and maintenance

Some of these steps will be repeated many times during the life of the system

Process Step: Feasibility

A **feasibility study** precedes the decision to begin a project.

- What is the scope of the proposed project?
- Is the project technically feasible?
- What are the projected benefits?
- What are the costs,?
- Are the resources available?
- What are the risks and how can they be managed?

A feasibility study leads to a **decision**: go or no--go.

Process Step: Requirements

Requirements define the function of the system from the client's viewpoint.

The requirements establish the system's functionality, constraints, and goals by consultation with the client, customers, and users.

They must be developed in a manner that is understandable by both the client and the development staff.

Process Step: Requirements

This step is sometimes divided into:

- Requirements analysis
- Requirements definition
- Requirements specification

(SRS document : The system shall allow a student to borrow a maximum of 3 books for 14 days.)

The requirements may be developed in a self--contained study, or may emerge incrementally.

Failure to agree on the requirements and define them adequately is one of the biggest cause of software projects failing.

Process Step: System Design

Design describes the system from the software developers' viewpoint

System design:

Establish a system architecture, both hardware and software, that matches the requirements

Program design:

Represent the software functions in a form that can be transformed into one or more executable programs

Preliminary **user testing** is often carried out as part of the design step.

Process Step: Implementation

Implementation (coding)

The software design is realized as a set of programs or program units.

These software components may be written by the development team, acquired from elsewhere, or modified from existing components.

Program testing

Program testing by the **development staff** is an integral part of implementation.

Individual components are tested against the **design**.

The components are integrated and tested against the **design** as a complete system.

Process Step: Acceptance and Release

Acceptance testing

The system is tested against the **requirements** by the **client**, often with selected customers and users.

Delivery and release

After successful acceptance testing, the system is delivered to the client and released into production or marketed to customers.

Process Step: Operation and Maintenance

Operation:

The system is put into practical use.

Maintenance:

Errors and problems are identified and fixed.

Evolution:

The system evolves over time as requirements change, to add new functions, or adapt to a changing technical environment.

Phase out:

The system is withdrawn from service.

This is sometimes called the Software Life Cycle

Sequence of Process Steps

Every software project will include these basic processes, in some shape or form, but:

- The steps may be formal or informal
- The steps may be carried out in various sequences

Sequence of Process Steps

Major alternatives

In this course, we will look at three categories of software development processes:

Sequential:

As far as possible, complete each process step before beginning the next. **Waterfall model.**

Iterative:

Go quickly through all process steps to create a rough system, then repeat them to improve the system. **Iterative refinement.**

Incremental:

A variant of iterative refinement in which small increments of software are placed in production (sprints). **Agile development.**

Heavyweight and Lightweight Software Development

In a **heavyweight process**, the development team works through the process steps slowly and systematically, with the aim of fully completing each process step and deliverables for that step that will need minimal changes and revision.

Example: **Modified Waterfall Model**

In a **lightweight process**, the development team releases working software in small increments, and develops the plans incrementally, based on experience. Each increment includes all the process steps. There is an expectation that changes will be made based on experience.

Example: **Agile Software Development**

Deliverables

Deliverables

A deliverable is some work product that is delivered to the client.

- In a **heavyweight process**, each process step creates a deliverable, usually documentation, e.g., a requirements specification.
- In a **lightweight process**, the deliverables are incremental working code, with minimal supporting documentation.

